Is a Cached Token Worth It? Measuring the Energy Discount of Prefix Caching in LLM Serving

Midterm Report

Aagam Sachin Bothara
Department of Computer Science, University of Illinois Chicago
aboth@uic.edu

Abstract—On an A100, a cached prefix token costs under 0.4% of a recomputed one, and enabling SGLang's radix cache costs nothing measurable when nothing is reused; output length, not prefix length, decides what fraction is saved. On held-out ShareGPT, agent, and RAG replays, cache-unaware energy accounting errs by 29–39%; every cache-aware variant, by about 6–10%.

I. Project Type and Status

An individual research project; the questions are unchanged. RQ1: how much GPU energy does reusing a cached prefix actually save in SGLang [3]? RQ2: does an accounting model with an explicit cached-token cost predict energy better than cache-unaware, pricing-inspired, and FLOP-informed baselines? Prior energy studies compare cache settings [1], [2]; this project separates the cost of reuse from the cost of having the cache enabled, and tests whether accounting models need a cached-token cost at all. Status at Nov 1. All planned measurements are done, including November's: the $D(L)$ grid, a second model and GPU, the zero-hit control, the vLLM reproduction, and the held-out RQ2 evaluation. What remains is one open RQ2 discrepancy, figures, the presentation, and the final report. The main change, answering feedback that cache on vs. off was unclear, is a precise definition of the conditions (Section II).

II. Clarifying the Experimental Conditions

"Cache on" vs. "off" conflates whether the radix cache is enabled, whether a request's prefix is actually found in it, and side effects of the flag itself; Table I separates them. The per-request KV cache used in decoding stays on throughout; only cross-request reuse changes.

TABLE I
Measurement conditions. L = shared-prefix length; last column is the expected cached_tokens per request. Every window, in every condition, starts with one excluded warm-up request.

Name	Radix cache	Prefix contents	Expected
Off	disabled	identical across requests	0
Miss	enabled	first token unique per request	0
Hit	enabled	identical; warm-up inserts it	L

Primary contrast: Hit vs. Miss. Both run on the same server process with the same flags and token counts; only whether the prefix is found differs, so their gap is the value of reuse alone. I redefine the discount as

$$D(L) = \frac{E_{\text{miss}}(L) - E_{\text{hit}}(L)}{E_{\text{miss}}(L) - E_{\text{miss}}(0)}, \tag{1}$$

where E is mean active GPU energy per request, and suffix length, output length, and concurrency are held fixed. $E_{\text{miss}}(0)$ is measured with the prefix removed. I also report energy saved per reused token, $(E_{\text{miss}}(L) - E_{\text{hit}}(L))/L$.

Secondary contrast: Miss vs. Off. The zero-hit control, $\Delta_{\text{on}}(L) = E_{\text{miss}}(L) - E_{\text{off}}(L)$, is what enabling the cache costs when nothing is reused (insertion, matching, eviction), the effect Zine et al. [2] saw on a workload with no repeated prefixes.

Confounders held fixed. (1) Scheduling policy. With the radix cache disabled, SGLang silently replaces cache-aware policies such as LPM with FCFS (_validate_and_adjust_policy), so Off vs. Hit could compare two schedulers. I pin --schedule-policy fcfs everywhere. (2) Exact token counts. Requests are token IDs sent to /generate with no chat template; each suffix starts with a per-request unique token, so no spurious 1-token hits. (3) Fixed work. ignore_eos with fixed max_new_tokens, greedy decoding, fixed chunked-prefill size, page size 1, CUDA graphs on. (4) Cache state. /flush_cache before every window, then one excluded warm-up request in every condition (in Hit it inserts the prefix).

Verification. A window is accepted only if every response's cached_tokens matches Table I. In all 1,016 synthetic windows (463,572 requests, both models, GPUs, and engines), every request matched.

III. Measurement Setup as Built

Platform. Chameleon Cloud had no A100 free in late October and its GPUs were reserved Nov 8–15, during the measurements planned for November, so the study runs on rented whole RunPod A100 80 GB SXM4 GPUs (SGLang 0.5.21). All energy is GPU energy from NVML; CPU, host memory, and the rest of the server are not measured. In every A100 session, NVML's energy counter agreed with its instantaneous power reading at idle to within 0.5 W. Clocks and power limit cannot be locked on a rented pod, so both are logged per window, and a fixed reference window opens every session. RunPod's A100 SXM4 pods

come with either a 400 W or a 500 W limit; all A100 results below are from 400 W cards (a Qwen3 session on a 500 W card, with a 60.8 J reference window, was rerun). Models: Qwen2.5-7B-Instruct and Qwen3-8B, BF16; second GPU: A40 (48 GB).

Harness. A closed-loop load generator holds c requests in flight over windows of at least 40 requests and 30 s (10 requests for 512-token outputs at c=1, where one takes 5–6 s), reading NVML's cumulative energy counter at window boundaries. Active energy subtracts idle power (60 s with the model loaded, before each block) times duration. Run order is randomized; each window logs SM clock, temperature, power limit, and other GPU processes. It also drives vLLM 0.11.0 and replays held-out workloads open-loop. Code and results: https://github.com/Aagam-Bothara/prefix-cache-energy.

IV. RQ1 Results

Noise floor. Within a session, the reference Miss window (L=2048, out 1, c=1) varied by 0.3–0.9% (CV of active energy per request); session means stayed within 45.4–45.9 J (Qwen2.5) and 50.0–50.4 J (Qwen3) across all 400 W sessions.

Zero-hit control. In the first sweep, Miss and Off ran on two server processes, and Δ_{on} stayed within +2.7% of E_{off}. To separate this from restart effects, cache-on and cache-off servers then alternated over 8 ABBA rounds, each a fresh server with its own idle measurement (Welch t-intervals over round means), at $L \in \{0, 2048, 8192\}$ with (out 1, c=1) and (out 128, c=4). $|\Delta_{\text{on}}|$ was at most 0.1% on SGLang and 0.3% on vLLM, and every 95% CI on both engines includes zero, so the "turn caching on" saving equals the reuse saving; the earlier +2.7% was most likely a restart effect. Filling the cache is free too: with 128- or 512-token outputs at c=1, the Hit warm-up request that inserts the prefix cost within 1% of a Miss warm-up at the same L (1532 vs. 1533 J at L=8192, out 512). A cached prefix therefore pays for itself on its first reuse.

Discount curve. With 1-token outputs, which isolate prefill, E_{miss} grows from 15.6 J (L=512) to 172.3 J (L=8192) while E_{hit} stays at 6.7–7.4 J, so D rises from 0.986 [0.980, 0.992] to 0.995 [0.995, 0.996] (Table II, first column). The saving is far outside the 0.9% noise floor at every L. A pricing rule of $b = 0.1a$ [4], [5] predicts $D \approx 0.9$; the measured values are above this, since a cached token cost about 0.08 mJ against about 20 mJ for a recomputed one. The 20 mJ is physically plausible: a fresh token needs about 14 GFLOP (2×6.5 B parameters, plus attention), so even at the A100's 312 TFLOPS BF16 peak, at these windows' 310 W active power, it would cost about 14 mJ; at $L \geq 2048$ the windows ran at 190–220 TFLOPS, 62–70% of peak. Longer outputs lower D, because decoding still reads the prefix at equal cost in Hit and Miss: D falls to 0.84–0.97 at 128 output tokens and 0.80–0.97 at 512. The energy saved per reused token stays between 0.016 and 0.028 J in every cell; D shrinks only because the rest of the request grows. When the saving becomes small: in Eq. 2 (Section V), reuse saves $(a - b)L$, while the denominator of Eq. 1 is $aL + d\,n_{\text{out}}L$, since every output token also reads the prefix, so $D \approx a/(a + d\,n_{\text{out}})$, roughly independent of L. The fitted a and d, pooled over all loads, put D below 0.9 beyond about 200 output tokens on Qwen2.5 and 110 on Qwen3. The formula omits load, since c_k cancels, but load matters: at 128 output tokens the saving per reused token barely changes from c=1 to c=16, while the rest of the denominator shrinks about threefold, so D is 0.84–0.91 at c=1 and 0.95–0.97 at c=16. Measured D was below 0.9 only at 128 tokens with c=1 and 512 with c=4.

Where the saving comes from. Mostly time, not power: at L=8192 with 1-token outputs, Miss and Off drew about 397 W at about 1260 MHz and Hit about 363 W at the full 1410 MHz, but each request finished far sooner. With 128 or 512 output tokens, all conditions held 1410 MHz at 345–394 W.

Second model: Qwen3-8B. At five load levels (out 1 at $c \in \{1, 4\}$; out 128 at $c \in \{1, 4, 16\}$), the pattern is the same: D is 0.979–0.997 with 1-token outputs and 0.80–0.96 with 128 output tokens. Each reused token saves 0.020–0.026 J, 6–29% more than on Qwen2.5-7B in the same cell (median 13%).

Second GPU: A40. At $L \in \{2048, 8192\}$ with (out 1, c=1) and (out 128, c=16), the A40's D matches the A100's to within 0.003, but the joules saved per reused token are 26–36% higher (0.024–0.029 J), so energy constants must be fitted per GPU.

vLLM reproduction. Following PrefixBench [1], I sent sequential requests sharing an 8192-token prefix (3 windows of ≥40 requests and ≥30 s) to one vLLM 0.11.0 server with prefix caching and one with --no-enable-prefix-caching, serving Qwen2.5-7B. This single pair was not repeated, so there is no CI over servers. Cache on with reuse vs. off saves 97.3% for 1-token outputs (176.2 J → 4.7 J per request) and 33.6% for 128-token outputs (541.6 J → 359.4 J). The per-token saving (0.021 J) and D (0.995 and 0.928) match SGLang closely. PrefixBench reports 443 vs. 337 J per request on an H100 (Qwen2-7B, same prefix), a 24% saving, without stating the output length, so the comparison is only qualitative. This run's Miss vs. Off gap (−0.4% to −1.7%) would echo Zine et al. [2], but alternating rounds put vLLM's Δ_{on} at −0.3% to 0.0%: most likely a restart effect too.

V. RQ2 Results: Accounting Models

Model. All four variants are Eq. 2 of the proposal,

$$\hat{E} = a\,n_{\text{fresh}} + b\,n_{\text{cached}} + c_k\,n_{\text{out}} + d\,n_{\text{out}}\,n_{\text{ctx}}, \quad (2)$$

with n_{ctx} the mean context during decoding (prompt plus half the output). They differ only in the prefill part: explicit fits a and b freely; cache-unaware forces $b = a$; pricing forces $b = 0.1a$; FLOP-informed replaces $a\,n_{\text{fresh}} + b\,n_{\text{cached}}$ by k times the remaining prefill

TABLE II

$D(L)$ across output length and concurrency: Qwen2.5-7B, A100 (400 W), mean of 3 windows per condition. CIs in the text are 95% t-intervals over window means (Welch degrees of freedom, delta method for D); 3 windows are too few for a bootstrap. †Not resolved: 10-request windows and 512 decode tokens leave CIs as wide as [0.49, 0.90] (L=2048). The out = 512, c=4 cell ran in a separate session; a repeat at $L \leq 1024$ (4 windows) gave 0.903 [0.70, 1.10] and 0.859 [0.77, 0.94].

	out 1			out 128			out 512		
L	c=1	c=4	c=16	c=1	c=4	c=16	c=1†	c=4	c=16
512	0.986	0.982	0.989	0.843	0.924	0.949	n.r.	0.795	0.903
1024	0.993	0.990	0.990	0.908	0.918	0.968	n.r.	0.973	0.921
2048	0.992	0.994	0.993	0.889	0.948	0.962	n.r.	0.933	0.915
4096	0.994	0.996	0.995	0.909	0.949	0.966	n.r.	0.865	0.903
8192	0.995	0.996	0.996	0.914	0.960	0.969	n.r.	0.872	0.913

TABLE III

RQ2 mean absolute error. Synthetic: leave one L out; held-out: 36 replay windows per model, fitted on synthetic data. The two 156.9% entries are a coincidence (156.86 and 156.90).

	Qwen2.5-7B		Qwen3-8B	
Variant	synthetic	held-out	synthetic	held-out
cache-unaware ($b = a$)	156.9%	29.2%	156.9%	38.9%
pricing ($b = 0.1a$)	25.8%	7.0%	23.1%	7.6%
FLOP-informed	7.0%	9.3%	10.2%	6.1%
explicit (a, b fit)	9.5%	10.3%	10.2%	6.4%

FLOPs. The load term, one c_k per fitted concurrency k interpolated in $\log k$, reflects batching sharing a decode step's weight reads; without it every variant errs by over 65%. Coefficients are fitted by least squares on synthetic windows only (387 over 9 load levels for Qwen2.5, 201 over 5 for Qwen3).

Held-out workloads. ShareGPT conversations (mean prompt 684 tokens, output 257); agent requests sharing one of four ∼3,000-token tool prompts (3,110, 73); and RAG questions over Zipf-popular LongBench documents (5,493, 21). Each has 3 windows of 120 requests, replayed open-loop from an empty cache with fixed-rate and Poisson arrivals at 1 and 4 requests/s: 36 windows per model, 56–96% of prompt tokens cached. A window's prediction sums per-request estimates, each at its own load.

Findings. (1) Ignoring the cache is clearly wrong. On held-out windows, explicit's error is 10.3% vs. 29.2% for cache-unaware on Qwen2.5 and 6.4% vs. 38.9% on Qwen3. Both give $p = 2/512 \approx 0.004$, the minimum for an exact paired sign-flip test over 9 replay blocks. This is RQ2's primary comparison; the tests among cache-aware variants below are exploratory. (2) A cached token's fitted cost is near zero: $b = -0.36$ mJ (95% bootstrap CI over windows $[-0.67, -0.10]$) against $a = 20.9$ mJ per fresh token on Qwen2.5, and $b = -0.07$ mJ ($[-0.33, +0.14]$) against $a = 23.4$ mJ on Qwen3. A negative cost is unphysical (other terms likely absorb a small effect), but $|b| < 2\%$ of a either way, and RQ1's direct 0.08 mJ is under 0.4%. A cached token is much closer to free than the 10% that API pricing commonly charges [4], [5]. (3) The cache-aware variants are close. On Qwen2.5, FLOP-informed beats explicit by 1.0 point ($p = 0.004$) and pricing beats it by 3.3 ($p = 0.020$); on Qwen3, neither difference is clear ($p = 0.44$ and 0.09). FLOP-informed has no cached-token parameter yet does at least as well as fitting b: counting the prefill FLOPs executed suffices, so a cached token's cost need not be measured. Synthetic errors are larger because 1-token Hit windows cost only a few joules. (4) Explicit and FLOP-informed underpredict RAG at 1 request/s by 19–30% on both models, against mostly 1–10% on agent and ShareGPT; pricing's cached-token charge happens to offset this, and a per-request constant does not remove it. My hypothesis: RAG reuses many cached documents; synthetic Hit windows reuse one.

VI. Changes from the Proposal

RQs, baselines, and Eq. 2 are unchanged. (1)–(2) The conditions and controls of Section II. (3) The platform moved from Chameleon Cloud to RunPod A100s, for the reason given above; RunPod has no A30, so the second GPU is an A40. (4) Qwen3-8B replaces Llama-3.1-8B-Instruct: close in size, no gated-model access. (5) NVML's energy counter is read directly instead of through llmtrace, which samples power and supports only vLLM.

VII. Plan to December 6 and Risks

Nov 2–15: test the RAG hypothesis with a synthetic cell reusing one of many cached documents; figures ($D(L)$ by output length and load; held-out error by workload). Nov 16–Dec 3: harness documentation; presentation. Dec 4–6: final report. Risks and limitations. An unexplained RAG gap will be reported per workload, not tuned away on held-out data. Synthetic windows never batch hits with misses; the replays do, but that effect is not isolated. Synthetic prompts are random token IDs, and power depends partly on the data, so joules per token on natural text may differ (the real-text replays partly cover this).

Use of AI Tools

Claude (Anthropic) was used to help structure this report, check SGLang's scheduling code for cache-dependent behavior, edit language, and write and debug parts of the measurement harness and analysis scripts, including a mock server for testing them. All measurements and numbers come from my own runs.

References

[1] O. Shewale, D. Kumar, and D. K. Yadav, "PrefixBench-H100: Characterizing prefix reuse and time-to-first-token in H100 LLM serving," arXiv:2609.19657, 2026.

[2] N. Zine et al., "Attention to detail: Evaluating energy, performance, and accuracy trade-offs across vLLM configurations," arXiv:2607.09172v2, 2026.

[3] L. Zheng et al., "SGLang: Efficient execution of structured language model programs," in Proc. NeurIPS, vol. 37, 2024.

[4] OpenAI, "API pricing," https://openai.com/api/pricing/, accessed Oct. 2026.

[5] Anthropic, "Prompt caching," Claude API documentation, https://platform.claude.com/docs/en/build-with-claude/prompt-caching, accessed Oct. 2026.